Variability-Aware Analysis of Kbuild Makefiles

ANONYMOUS AUTHOR(S)

The Linux kernel is one of the most configurable software systems, and its `kbuild` Makefiles control how thousands of configuration options determine which source files are compiled and linked into the kernel build. To understand why a source file is omitted from a build, or which configuration options enable it, a developer needs its build condition: a logical formula over configuration options that specifies when the file is built. Invoking Make on a single configuration reveals only the files compiled for that configuration, while exhaustively enumerating all configurations is intractable. Standard symbolic execution spawns new execution paths at every conditional branch and suffers from path explosion, whereas prior family-based static analyses of Kbuild Makefiles enumerate complete string definitions for each variable. We present KFOLD, a symbolic execution approach for `kbuild` Makefiles that maintains a consolidated symbolic state mapping each Make variable to a set of guarded words, where each word is paired with a path constraint over configuration options in the style of a value summary. Append statements add guarded words incrementally, overwrites perform guarded updates that exclude the new path constraint from existing words, and state merging combines the symbolic states of conditional branches at join points, keeping the symbolic state proportional to the number of distinct words in each variable. We represent the resulting build conditions as quantifier-free first-order formulae and use an SMT solver to check their satisfiability and generate valid configurations.

CCS Concepts: • **Software and its engineering → Software verification and validation**.

Additional Key Words and Phrases: Kbuild, Makefile, symbolic execution, variability, Kconfig

1 Introduction

2 Background

3 Motivating Example

This section walks through the symbolic execution of a Kbuild Makefile in KFOLD to illustrate how build conditions are computed and how the consolidated symbolic state remains compact. At a high level, the Linux kernel is built in two steps: configuring and building [5]. In the configuration step, a developer selects features using a configurator that reads Kconfig files, which specify configuration options and their dependencies, and writes the chosen settings to a `.config` file. Boolean options take the value `y` to include a feature or remain unset to exclude it, while tristate options take `y` (built directly into the kernel), `m` (compiled as a loadable kernel module), or unset. In the build step, Kbuild uses GNU Make to evaluate Makefiles and determine which source files to compile and link. Programmers encode variability in Kbuild Makefiles by populating reserved variables known as goal definitions: `obj-y` lists object files built into the kernel image and `obj-m` lists object files built as modules. Makefiles support runtime variable name construction [3]: a statement such as `obj-$(CONFIG_EXT4_FS) += ext4.o` expands the configuration option on the left-hand side to form the target variable name `obj-y`, `obj-m`, or `obj-` (which Kbuild ignores).

Fig. 1 shows a Kbuild Makefile adapted from the Linux `ext4` file system. A developer who modifies a source file in this directory and runs a build may ask why `ext4.o` was omitted from the build, or which configuration settings would compile it. Running Make with a single `.config` file answers this question for one configuration at a time. A build condition summarizes this behavior across the configuration space as a logical constraint over configuration options that specifies when the file is included in the build.

Let E_y and E_m denote the predicates `CONFIG_EXT4_FS=y` and `=m`, T_y and T_m denote the corresponding settings of `CONFIG_EXT4_KUNIT_TESTS`, and V denote `CONFIG_FS_VERITY=y`. KFOLD treats configuration options as symbolic inputs and simulates the evaluation of the Makefile over a single consolidated state. Each Make variable is mapped to a variational set of words [8] in which

```
1  obj-$(CONFIG_EXT4_FS) += ext4.o
2  obj-$(CONFIG_EXT4_KUNIT_TESTS) := ext4-inode-test.o
3  ifeq ($(CONFIG_FS_VERITY),y)
4    EXTRA := verity
5  else
6    EXTRA := crypto
7  endif
8  obj-y += $(EXTRA).o
9  ext4-inode-test-y += inode-test.o
10
```

Fig. 1. Makefile adapted from Linux `fs/ext4/Makefile`. Only line 1 is verbatim; lines 2 and 9 are adapted and lines 3–8 are constructed to add an overwrite, a conditional, and a composite object.

Table 1. Build conditions of the files built by Fig. 1.

File	Built into the kernel (`obj-y`)	Built as a module (`obj-m`)
`ext4.o`	$E_y \wedge \neg T_y$	$E_m \wedge \neg T_m$
`ext4-inode-test.o`	T_y	T_m
`inode-test.o`	T_y	T_m
`verity.o`	V	—
`crypto.o`	$\neg V$	—

each word is paired with a guard over configuration options, so the membership of each word forms a Boolean value summary in the sense of MultiSE [6]. Line 1 expands the left-hand side and appends `ext4.o` to `obj-y` under guard E_y and to `obj-m` under guard E_m. Line 2 is a conditional overwrite whose left-hand side also depends on a tristate option: under T_y it overwrites `obj-y` with `ext4-inode-test.o`, and under T_m it overwrites `obj-m`. To preserve existing words on paths where the overwrite does not take effect, KFOLD performs a guarded update [3, 6]: it adds `ext4-inode-test.o` to `obj-y` under T_y and to `obj-m` under T_m, while conjoining the negated guard with the prior entries so that `ext4.o` remains in `obj-y` under $E_y \wedge \neg T_y$ and in `obj-m` under $E_m \wedge \neg T_m$. Lines 3–7 assign `EXTRA` in both branches of an `ifeq` conditional; at the join point, state merging combines the two branches into a single symbolic state mapping `EXTRA` to `verity` under V and `crypto` under $\neg V$. Line 8 expands `$(EXTRA)` and concatenates each alternative with `.o`, appending two guarded words with complementary guards to `obj-y`. Finally, line 9 defines `inode-test.o` as a member of the composite object `ext4-inode-test.o`, which inherits the build condition of its parent composite. Tab. 1 lists the resulting build conditions. These conditions are local to `fs/ext4/Makefile`; the full build condition of each file conjoins the condition under which Kbuild descends into `fs/ext4/`.

Representing build conditions as quantifier-free first-order formulae allows KFOLD to answer developer queries directly with a satisfiability modulo theories (SMT) solver. Evaluated against a concrete `.config`, the formula for `ext4.o` explains why the file was excluded from the build, *e.g.*, because T_y holds and overwrote `obj-y`. Conjoined with the feature-model constraints extracted from Kconfig, the formula enables an SMT solver to find a satisfying assignment that compiles `ext4.o` by satisfying $E_y \wedge \neg T_y$. KFOLD exposes these two analyses through the `why` and `config-for` commands; § 6 validates the solver-generated configurations with the kernel configurator and concrete builds.

Prior symbolic execution of Kbuild Makefiles by Nguyen and Nguyen [5] follows conventional path-based exploration: it forks a new execution state at every conditional or multi-valued expansion and mitigates path explosion by combining execution paths and applying a split-merge strategy [1, 6]. On Fig. 1, expanding the two tristate options and branching on the conditional spawns $3 \times 3 \times 2 = 18$ execution paths before merging, and each subsequent independent conditional can double the path count. By contrast, KFOLD maintains a single consolidated symbolic state in which `obj-y` and `obj-m` hold six guarded words in total, and each subsequent append adds only the new guarded words. MultiSE [6] introduced value summaries to share symbolic expressions across paths without auxiliary variables, and Kmax [3] similarly localizes configuration explosion to individual statements by tracking reaching configurations over complete strings.

In summary, representing variables as sets of guarded words and merging states at conditional join points yields per-file build conditions at a cost proportional to the number of distinct words, enabling constraint-based reasoning over the build system.

4 Approach

4.1 Symbolic State and Algorithm

This subsection defines the symbolic state representation and the top-level worklist algorithm that traverses the Makefile hierarchy of a project. Let M be a collection of Kbuild Makefiles and c be a configuration that assigns values to the configuration options referenced in M. For each target object o, KFOLD computes a quantifier-free formula $\Phi_M(o)$, its build condition, such that evaluating $\Phi_M(o)$ under c predicts whether Kbuild compiles o under c. The symbolic state σ is an abstract symbol table that maps each Make variable n to a variational set of guarded words [8]: an entry $w \mapsto \phi$ records that the word w is a member of n along all execution paths whose configuration satisfies the guard ϕ, a Boolean formula over configuration options, analogous to a guarded symbolic expression in a value summary [6]. We write $\sigma[n](w) = \bot$ when w is absent from $\sigma[n]$, and we write $\sqcup$ for the value-summary union of two guarded word sets, which coalesces duplicate words by taking the disjunction of their guards and drops entries with unsatisfiable guards [6]. Unlike value summaries over scalar variables whose guards are mutually exclusive, the guards of distinct words in $\sigma[n]$ may overlap because a single configuration can include multiple space-separated words in a Make variable. For an individual Makefile, EXEC (Alg. 2) evaluates its statements in a single pass starting from an empty symbolic state under the ambient guard $\top$; restricting a state to a guard δ conjoins δ with the guard of every word in the state.

Alg. 1 takes a project's entry Makefiles E and goal-definition prefixes Γ, and returns the build-condition map Φ. It maintains a worklist Q of pairs (m, γ), where γ is the directory reachability guard under which Kbuild descends into Makefile m, together with a table $R[m]$ recording the disjunction of reachability guards already explored for m. In each iteration (line 2), the algorithm dequeues (m, γ), restricts attention to the unexplored configurations $\delta = \gamma \wedge \neg R[m]$ (line 4), and skips the visit if δ is unsatisfiable (line 5). Otherwise, it symbolically executes m and restricts the resulting state S to δ (line 6), updates the explored guard $R[m]$ (line 7), enqueues each subdirectory referenced in S along with its guard (line 9), and accumulates the build conditions of all objects selected via Γ (line 11). When multiple Makefiles or directory routes select the same object o, the disjunction at line 11 combines their conditions. For Fig. 1, the symbolic state S after line 6 contains the guarded words of `obj-y` and `obj-m` from Tab. 1 conjoined with the reachability guard of the `fs/ext4/` directory.

Alg. 1. Build conditions of the objects of a source tree

Input: entry Makefiles E with reachability guards, goal-definition prefixes Γ
Output: map Φ from objects to build conditions

$R[m] \leftarrow \bot$ for each Makefile m; $Q \leftarrow E$; $\Phi[o] \leftarrow \bot$ for each object o;
while Q *is nonempty* **do**
  remove (m, γ) from Q;
  $\delta \leftarrow \gamma \wedge \neg R[m]$;
  if δ *is unsatisfiable* **then continue**;
  $S \leftarrow$ state of m after Exec, restricted to δ;
  $R[m] \leftarrow R[m] \vee \gamma$;
  foreach *subdirectory* d *with guard* η *in* S **do**
    add (Makefile of d, η) to Q;
  foreach *object* (o, ϕ) *selected by* S *through* Γ **do**
    $\Phi[o] \leftarrow \Phi[o] \vee \phi$;
return Close(Φ); `// objects that rules need`

Alg. 2. Exec: symbolic execution of one statement

Input: statement s, state σ, ambient guard g
Output: updated state σ

if s *is an assignment* x *op* e **then**
  $V \leftarrow$ the words $w \mapsto \phi_w$ of e (or $e \mapsto \top$ if the variable is recursively expanded);
  foreach *name* n *with guard* ψ_n *in the expansion of* x **do**
    $\psi \leftarrow g \wedge \psi_n$;
    if *op is an overwrite or* $\sigma[n]$ *is undefined* **then**
      $\sigma[n] \leftarrow \{w \mapsto \psi \wedge \phi_w\} \sqcup \{w \mapsto \phi \wedge \neg\psi : (w \mapsto \phi) \in \sigma[n]\}$;
    else
      $\sigma[n] \leftarrow \sigma[n] \sqcup \{w \mapsto \psi \wedge \phi_w\}$;
else if s *is a conditional with test* t*, then-arm* T*, else-arm* F **then**
  $\kappa \leftarrow$ condition under which t holds; $\sigma_t, \sigma_e \leftarrow$ copies of σ;
  Exec$(T, \sigma_t, g \wedge \kappa)$; Exec$(F, \sigma_e, g \wedge \neg\kappa)$;
  foreach *name* n *assigned in* T *or* F *and word* w **do**
    if $\sigma_t[n](w) = \sigma_e[n](w) = \phi$ **then**
      $\sigma[n](w) \leftarrow g \wedge \phi$;
    else
      $\sigma[n](w) \leftarrow (g \wedge \kappa \wedge \sigma_t[n](w)) \vee (g \wedge \neg\kappa \wedge \sigma_e[n](w))$;
return σ;

4.2 Assignments

This subsection explains how KFOLD symbolically executes variable assignments, including runtime variable name construction, guarded updates on overwrites, and the two Make variable flavors. Alg. 2 evaluates each statement under an ambient guard g representing the conjunction of enclosing branch conditions, which Kmax calls the reaching configuration [3]. Because Make allows variable names on the left-hand side of an assignment to be computed dynamically via variable expansion, as in `obj-$(CONFIG_EXT4_FS)`, line 3 expands the left-hand expression x into concrete variable

Table 2. Make constructs and their interpretation.

Construct	Interpretation	Boundary
`+=`, `=`, `:=`	Guarded words; the flavor decides when the right-hand side is expanded	Word order and duplicates are not kept.
`ifeq`, `ifneq`, `ifdef`, `ifndef`	Both arms are executed and the states are merged	—
Variable references	Guarded words, or the empty word when none is present	—
`include`	The directive is replaced by the statements of the file	A path that cannot be resolved statically contributes nothing.
Rules and sub-makes	Prerequisites are followed from selected objects	Other recipes are not executed.
`$(shell ...)`	A concrete subprocess	The host environment can change the result.

names n paired with their guards ψ_n: `obj-y` under E_y, `obj-m` under E_m, and `obj-` when the option is unset. A single assignment statement can thus update multiple target variables under different effective guards $\psi = g \wedge \psi_n$. An append assignment (`+=`) unions the right-hand guarded words into $\sigma[n]$, conjoining each word's guard ϕ_w with ψ (line 8). An overwrite assignment (`:=` or `=`) replaces the variable's contents only on paths where ψ holds: line 6 performs a guarded update [3, 6] that assigns guard $\psi \wedge \phi_w$ to each new word while restricting every pre-existing word $(w \mapsto \phi) \in \sigma[n]$ to $\phi \wedge \neg\psi$. In Fig. 1, line 2 overwrites `obj-y` under $\psi = T_y$, strengthening the guard of `ext4.o` from E_y to $E_y \wedge \neg T_y$. Conjoining $\neg\psi$ ensures that `ext4.o` is excluded under $E_y \wedge T_y$, matching concrete Make evaluation where the second assignment replaces the earlier value.

KFOLD also models the two variable flavors in GNU Make, following the distinction in SYMake [7] and Kmax [3]. A simply-expanded variable (`:=`) evaluates its right-hand side immediately at assignment time. A recursively-expanded variable (`=`) stores its right-hand side verbatim and defers evaluation until the variable is referenced; accordingly, KFOLD stores an unexpanded expression when a recursive definition contains variable references (line 2) and expands deferred references in goal definitions after executing the Makefile. Linux's `drivers/xen/Makefile` illustrates why deferred expansion is needed: it appends `$(xen-pad-y)` to `dom0-y` one line before defining `xen-pad-y`, so only expansion at reference time resolves `xen-acpi-pad.o`. Tab. 2 summarizes the Make constructs that KFOLD interprets and their analysis boundaries.

4.3 Expansion of Names and Words

This subsection describes how KFOLD lifts string concatenation, variable expansion, and file inclusion to sets of guarded words. In Make, the right-hand side of an assignment is whitespace-delimited into independent word expressions. Adjacent subexpressions within a single token, such as `$(EXTRA).o` on line 8 of Fig. 1, are concatenated using a lifted cross-product operation [3] that pairs each concrete string combination with the conjunction of their guards, yielding `verity.o` under V and `crypto.o` under $\neg V$. Treating a whitespace-separated list such as `$(a-y) $(b-y) $(c-y)` as a set of independent words avoids taking the cross product across the three variables, preserving each word's guard directly. When a variable reference may be undefined along some paths, KFOLD includes the empty string guarded by the negation of the variable's definedness condition, which is essential when the reference appears inside a word or list. For instance, Linux's `dvb-core` Makefile defines `dvb-net-$(CONFIG_DVB_NET) := dvb_net.o` and includes `$(dvb-net-y)` in `dvb-core-objs`; retaining the empty alternative ensures that the remaining members of `dvb-core-objs` do not spuriously acquire a dependency on `CONFIG_DVB_NET`. Built-in text functions and substitution

references such as `ixgbe-$(CONFIG_FCOE:m=y)` are lifted similarly by applying the string transformation to each word while preserving its guard. Finally, KFOLD inlines `include` directives whose target paths resolve statically and slices away variable assignments that cannot flow into goal definitions or build rules.

4.4 State Merging at Conditionals

This subsection presents how KFOLD evaluates conditional blocks and merges branch states at join points. For an `ifeq` or `ifneq` directive, KFOLD expands both operands and computes the branch guard κ as the disjunction of conjunctions over matching (or non-matching) guarded words; for `ifdef` and `ifndef`, κ is the disjunction of guards under which the variable has a non-empty definition [3]. Chains of `else ifeq` directives are desugared into nested conditionals in the else-arm. To evaluate a conditional under ambient guard g, KFOLD clones the incoming state σ into σ_t and σ_e (line 10), executes the then-arm under $g \wedge \kappa$ and the else-arm under $g \wedge \neg\kappa$, and merges the resulting states at the confluence point (line 16), following static state merging [1, 4]. Only variables modified in either branch are iterated over during the merge (line 12). In Fig. 1, the then-arm assigns `verity` under V to `EXTRA` and the else-arm assigns `crypto` under $\neg V$, so the merged state maps `EXTRA` to both guarded words and execution proceeds with a single symbolic state. Whenever a word w of a modified variable n carries the identical guard ϕ in both branches, KFOLD simplifies its merged guard directly to $g \wedge \phi$ (line 14). This simplification is critical for scalability: in a Makefile with k sequential conditionals that each append a distinct object to `obj-y`, preserving unchanged guards avoids wrapping every previously added word in k nested case splits.

PROPOSITION 4.1. *Let σ' be the symbolic state that Alg. 2 produces after a conditional statement with branch guard κ under ambient guard g. For every configuration c satisfying $g \wedge \kappa$, σ' evaluates to the same word set as the then-arm state σ_t, and for every configuration c satisfying $g \wedge \neg\kappa$, σ' evaluates to the same word set as the else-arm state σ_e.*

Prop. 4.1 holds for the core calculus of assignments and conditionals in Alg. 2, where word order and multiplicity are abstracted. The proof follows from line 16 by distributing evaluation under c across the disjunction $(g \wedge \kappa \wedge \sigma_t[n](w)) \vee (g \wedge \neg\kappa \wedge \sigma_e[n](w))$; when $\sigma_t[n](w) = \sigma_e[n](w) = \phi$, factoring out ϕ yields $(g \wedge \kappa \vee g \wedge \neg\kappa) \wedge \phi \equiv g \wedge \phi$ (line 14).

4.5 Descending into Subdirectories

This subsection describes how KFOLD tracks reachability guards across recursive directory traversal. In Kbuild, listing a directory in a goal definition, *e.g.*, `obj-$(CONFIG_X) += sub/`, instructs Make to descend into `sub/` and evaluate `sub/Makefile`, paralleling the hierarchical directory structure of the source tree [3]. KFOLD propagates the guard of the directory entry as the incoming reachability guard γ for the child Makefile, so every object and subdirectory in the child inherits its parent's reachability constraint. A subdirectory Makefile can be reached along multiple directory routes. For instance, if a parent Makefile selects `a/` under guard A and `b/` under guard B, and both `a/Makefile` and `b/Makefile` select `shared/`, then an object `x.o` unconditionally listed in `shared/Makefile` has the build condition $A \vee B$. Simply marking `shared/Makefile` as visited after traversing `a/` would miss the reachability path through `b/`. To handle multiple routes and cycles, KFOLD records for each Makefile m the cumulative guard $R[m]$ of configurations already explored (line 7); when (m, γ) is dequeued, it computes the residual guard $\delta = \gamma \wedge \neg R[m]$ (line 4). If δ is unsatisfiable, no new configurations reach m and the entry is pruned (line 5); otherwise, KFOLD evaluates m under δ and disjoins γ into $R[m]$. Consequently, any cyclic directory reference terminates as soon as its residual guard becomes unsatisfiable, and because both the set of Makefiles and the configuration space are finite, $R[m]$ can be weakened only finitely many times before the worklist empties.

Because top-level project Makefiles primarily contain imperative build recipes and environment setup, KFOLD starts traversal from the entry Makefiles specified in the project settings (§ 5).

4.6 From Words to Objects

This subsection details how KFOLD resolves the guarded words of goal definitions into compiled object files according to Kbuild's linking and prerequisite rules. While goal definitions specify which compilation units and composites are selected, Kbuild's canned build rules determine which physical `.o` files are emitted. KFOLD inspects variables whose names match the goal-definition prefixes (*e.g.*, `obj-`, `lib-`) configured for the project, suffixed by `y`, `m`, or the empty suffix. When a goal definition selects a composite object t`.o`, Kbuild resolves its component member objects from the variables t`-y` and t`-objs`, and additionally from t`-m` when t is built as a module. Each member object is compiled under the conjunction of its own membership guard and the build condition of the parent composite t, as illustrated by `inode-test.o` in Tab. 1. Composite expansion accounts for a large fraction of the kernel build: in Linux, 16,336 of the 32,829 extracted objects are members of composite objects. Beyond basic goal definitions and composites, KFOLD models three Kbuild semantics that govern object emission.

Built-in composite objects. When a composite object is selected via `obj-y` to be linked directly into the kernel image, Kbuild compiles its member objects and links them into the parent archive without emitting an intermediate t`.o` file; accordingly, KFOLD emits the composite object t`.o` itself only under the guard for which it is selected as a module via `obj-m`.

Routes into directories. Kbuild compiles the built-in (`obj-y`) objects of a subdirectory only when the subdirectory is reached through a chain of `obj-y` directory entries (tracked by Kbuild's internal `need-builtin` flag); if a parent Makefile enters the subdirectory solely via `obj-m`, only module targets in that subdirectory are built. KFOLD therefore tracks separate built-in and module reachability guards for each Makefile.

Rule prerequisites. Certain object files, such as linker scripts, generated offsets, and boot stubs, are never listed in goal definitions and are compiled solely because an explicit Makefile rule or sub-make command depends on them. During symbolic execution, KFOLD records explicit rules (mapping expanded targets to prerequisites under their ambient guards [3, 7]) as well as sub-make invocations that specify a target directory and goal; after worklist traversal, CLOSE (line 12) computes the transitive closure of prerequisites starting from all selected objects and top-level link targets such as `vmlinux.o`.

4.7 Using Build Conditions

This subsection describes how KFOLD formulates developer build queries as SMT constraint-solving problems over the extracted build conditions. Because each build condition $\Phi_M(o)$ is a quantifier-free first-order formula over symbolic configuration options, reasoning about when an object is built reduces to evaluating or solving $\Phi_M(o)$ with an SMT solver [5]. The `why` command takes an object file and a concrete `.config`, evaluates $\Phi_M(o)$ under that configuration's option assignments, and reports both the resulting build status and a condition chain highlighting which option clauses in $\Phi_M(o)$ are violated. The `config-for` command takes a commit or patch, maps the modified C and assembly source files to their corresponding target objects, conjoins their build conditions with the Kconfig feature-model constraints for the relevant options, and queries the SMT solver for a satisfying configuration that minimizes the number of option changes relative to a baseline `.config`. If two modified files require mutually exclusive configuration settings, the conjunction is unsatisfiable and KFOLD partitions the targets into compatible groups so each can be tested in a separate build.

4.8 Meaning of a Build Condition

This subsection states the domain encoding of configuration options and the soundness boundaries of the extracted build conditions. Each symbolic configuration option ranges over the values declared by its Kconfig type: a Boolean option ranges over {`y`, unset}, while a tristate option ranges over {`y`, `m`, unset} and conjoins the global module-support constraint `CONFIG_MODULES=y` with every `=m` guard. An extracted build condition $\Phi_M(o)$ is a static approximation of Kbuild's concrete behavior: KFOLD models the Kbuild subset of GNU Make shown in Tab. 2 and initiates traversal from the declared entry Makefiles, leaving arbitrary shell recipes and top-level orchestration rules outside the analysis. Moreover, $\Phi_M(o)$ captures the constraints encoded in Kbuild Makefiles alone; Kconfig inter-option dependencies (`depends on`, `select`, and `choice` blocks) are conjoined separately during solver queries (§ 4.7). Consequently, any satisfying assignment produced by the SMT solver is treated as a candidate configuration that must be validated by running the Kconfig configurator and executing a concrete build, as evaluated in § 6.

5 Implementation

This section describes the implementation of KFOLD, its project configuration format, and its command-line analyses. KFOLD is implemented in Python. Following Kmax [3] and Nguyen and Nguyen [5], it parses Makefiles into abstract syntax trees (ASTs) using `pymake3`, our Python 3 port of the Mozilla `pymake` parser, and represents guards and build conditions as quantifier-free formulae in the Z3 SMT solver [2]. Each configuration option is encoded as a finite enumeration sort in Z3 over two values ({`y`, unset} for the Boolean domain) or three values ({`y`, `m`, unset} for projects that enable the tristate domain). Guards are constructed from equality predicates over these option sorts using logical conjunction, disjunction, and negation, and KFOLD simplifies each formula eagerly after every Boolean operation [6].

During worklist traversal (Alg. 1), KFOLD queries Z3 to check guard satisfiability, prune infeasible directory routes, and restrict child states. To avoid redundant parsing and evaluation when a subdirectory Makefile is reached along multiple routes, KFOLD serializes the post-execution symbolic state of each Makefile—together with its recorded explicit rules—into a disk cache keyed by canonical file path; a subsequent visit under a new residual guard δ loads the cached state and conjoins δ directly (line 6 of Alg. 1). Because a Makefile can be re-evaluated under multiple residual guards across the directory hierarchy, the number of Makefile instances processed during a full-tree analysis can exceed the number of distinct Makefile files in the source tree. Alongside the build conditions, KFOLD records provenance metadata for each target object: the contributing Makefiles, goal definitions, and explicit rules, as well as the `include` directives resolved or left unresolved and any Makefiles that failed to parse.

Project-specific settings specify the entry Makefiles and Kbuild conventions of § 4.5 and § 4.6. Each project configuration declares the top-level entry directories (each optionally guarded by an enabling configuration option), the entry build goals, the recognized goal-definition prefixes and their aliases, fixed variable bindings such as `SRCARCH=x86`, the top-level link targets, the option domain (Boolean or tristate), and toggles for the three Kbuild linking semantics of § 4.6. We extract these declarations directly from each project's top-level Makefiles and annotate every entry with the source lines from which it is derived. For Linux x86, the project configuration specifies ten entry directories (three of them conditionally guarded), two entry goals, and fixed bindings for the architecture variables `SRCARCH` and `BITS` [3].

KFOLD provides a command-line interface that answers the developer queries of § 4.7 over a cached whole-tree analysis. The whole-tree cache stores the build condition, target classification, and provenance of every extracted object; it is keyed by a cryptographic digest of the analyzer

source and project settings, and tracks the timestamps of all read Makefiles and included files so that modifying any build file automatically invalidates the cache. Given an object file and a concrete `.config`, the `why` command reports a condition chain that traces the directory reachability route to the object's Makefile, the goal definition or composite object that selects the file, and the truth value of each clause under the given `.config`. When an object is omitted from the build, `why` pinpoints the first falsified conjunct in the condition chain and lists the configuration options responsible.

For the `config-for` command, KFOLD constructs a Kconfig feature model K using `kconfiglib` (extended to parse newer Kconfig syntax) and slices K to the cone of influence of the options referenced in the touched objects' build conditions. In K, each option is bounded above by its `depends on` constraints and below by incoming `select` clauses, and an option may be enabled only when activated by a `select`, an `imply`, a visible user prompt, or an active `default` attribute. Because K abstracts away the priority order among multiple active defaults, it over-approximates the set of valid Kconfig assignments. When invoked with verification enabled, `config-for` writes the solver's candidate assignment as a configuration fragment, runs the kernel configurator (`make olddefconfig`) to materialize a valid `.config`, and verifies both that the requested option settings are preserved and that KFOLD predicts every target object under the resulting configuration. Finally, the `blindspots` command evaluates the extracted build conditions across a corpus of configurations and reports build-system blind spots—source files that remain uncompiled under every configuration in the corpus—grouped by their subsystem entries in `MAINTAINERS`. The implementation of KFOLD and all evaluation scripts are available in the anonymized artifact.

6 Evaluation

We evaluate KFOLD to determine how accurately its build conditions predict the object files produced by concrete builds, how its consolidated state representation scales relative to path-forking symbolic execution and Kmax, how effectively its build conditions support developer maintenance tasks, and how each component of its design contributes to accuracy. Specifically, we investigate four research questions:

RQ1 (Build Agreement) When the extracted build conditions are evaluated under concrete configurations, how accurately do the predicted object sets match the ground-truth object files compiled by full builds?

RQ2 (Scalability) How do the running time and memory consumption of KFOLD scale with the number of conditional assignments compared with path-forking symbolic execution and Kmax, and what is the cost of analyzing entire project trees?

RQ3 (Developer Tasks) Can the extracted build conditions explain why source files are included or excluded in a build, synthesize valid configurations to compile files modified by real-world commits, and identify uncompiled blind spots across standard configurations?

RQ4 (Comparison and Ablation) How does KFOLD compare with Kmax in build agreement on the same concrete builds, and how much do `include` inlining, rule closure, and Kbuild link semantics contribute to KFOLD's precision and recall?

6.1 Experimental Setup

Subjects. Tab. 3 summarizes the five open-source Kbuild systems and concrete configurations used in our evaluation: Linux 7.2.8, BusyBox 1.38.0, Barebox 2026.09.0, Das U-Boot 2026.07, and coreboot 26.06. Linux and BusyBox are the two benchmark systems evaluated by Kmax [3] and Nguyen and Nguyen [5], allowing a direct comparison on established subjects. Barebox, Das U-Boot, and coreboot are widely used bootloader and firmware codebases whose Kbuild dialects introduce stage-specific goal-definition prefixes (*e.g.*, for secondary program loaders and ROM stages), testing whether

Table 3. Subjects, configurations, and the number of object files in each fresh build.

Subject	Version	Configuration	Objects
Linux	7.2.8	`tinyconfig` (i386)	528
Linux	7.2.8	`defconfig` (x86-64)	3,133
Linux	7.2.8	Debian (x86-64)	17,191
Linux	7.2.8	`allmodconfig` (x86-64)	29,625
BusyBox	1.38.0	`defconfig`	623
Barebox	2026.09.0	`sandbox_defconfig`	502
Das U-Boot	2026.07	`sandbox_defconfig`	1,344
coreboot	26.06	`qemu-i440fx`	626

KFOLD generalizes across Kbuild variants [3]. Each subject tree is unpacked from an official release tarball and verified against its published SHA-256 checksum where available (Linux, BusyBox, and coreboot). For BusyBox, we run `make gen_build_files` before analysis because BusyBox generates several of its Kbuild Makefiles from templates during build preparation. For Linux, we evaluate four representative configurations that span the configuration space: `tinyconfig` (i386) and `allmodconfig` (x86-64) represent the minimal and maximal kernel builds, `defconfig` is the standard default x86-64 kernel configuration, and the Debian x86-64 configuration represents a production distribution kernel [5]. Each of the remaining four subjects is evaluated on its standard default or sandbox configuration.

Ground-Truth Builds. Following Nguyen and Nguyen [5], we establish ground truth for each configuration c by running a clean concrete build and recording the compiled `.o` object files. Each configuration is built from scratch with `make -k -j8`, where `-k` instructs Make to continue compiling independent targets past any individual compile error so that the ground-truth inventory is complete. Build times range from 2.6 minutes for Linux `tinyconfig` to 100.2 minutes for Linux `allmodconfig`. The ground-truth object set B_c for configuration c comprises all `.o` files emitted in the build tree, excluding the post-processing `*.mod.o` stubs generated by `modpost`.

We modify stock configurations only when required by the host toolchain or missing external assets: in the Debian kernel configuration, we clear the trusted and revocation key-path strings because Debian's signing certificates are external to the upstream kernel tree, and `olddefconfig` drops four Debian-specific patch options; in Linux `allmodconfig`, we disable `WERROR`, `DRM_WERROR`, and `DRM_AMDGPU_WERROR` so warnings under GCC 16.2 do not abort compilation; in coreboot, we set `CONFIG_PAYLOAD_NONE=y` to avoid cloning the external SeaBIOS repository during the build; and in U-Boot, we pass `NO_PYTHON=1` because its device-tree Python wrapper invokes removed Python 2 APIs, which causes only the final `binman` image-packaging step to fail after all object files have been compiled. In BusyBox, `networking/tc.c` fails to compile against modern Linux kernel headers; we record such compile-time failures—along with downstream targets identified via `make -n` that Make skipped due to a failed prerequisite—separately from analysis false positives.

Metrics. For a subject codebase, KFOLD extracts a universe U of target objects and maps each object $o \in U$ to its build condition $\Phi_M(o)$. Substituting the concrete option settings of a configuration c into each condition yields the predicted object set $P_c = \{o \in U \mid c \models \Phi_M(o)\}$. When evaluating $\Phi_M(o)$ under c, any option that is omitted from `.config` or marked "`is not set`" takes the value unset; the Linux analysis uses the tristate option domain. Comparing the predicted set P_c against the ground-truth build inventory B_c, we measure true positives $TP = |P_c \cap B_c|$, false positives $FP = |P_c \setminus B_c|$, in-universe false negatives $FN_U = |(B_c \cap U) \setminus P_c|$, and out-of-universe built objects $O = |B_c \setminus U|$. From these counts we report precision $TP/(TP+FP)$ and overall build overlap $TP/|B_c|$. Whereas FN_U measures imprecision in the symbolic execution of extracted Makefiles (extracted

objects whose build condition evaluated to false despite being built), the overlap metric $TP/|B_c|$ penalizes the analysis for every built object outside the extracted universe U, such as arch-specific boot payloads under `arch/x86/boot/` invoked via top-level `bzImage` recipes.

Baselines and Ablations. We compare KFOLD against Kmax [3] (`kmaxall` v4.10) on Linux and BusyBox by evaluating Kmax's extracted Boolean conditions under the same concrete configurations and scoring them against B_c using the same metrics. In RQ2, we evaluate scalability on synthetic Kbuild Makefiles containing k independent conditional blocks, comparing KFOLD against a conventional path-forking symbolic executor (which forks a symbolic state at each conditional branch [5]) and against Kmax. In RQ4, we conduct an ablation study that disables three design components of KFOLD one at a time: static `include` inlining (`no_includes`), transitive prerequisite closure over explicit rules (`no_rules`), and the three Kbuild linking and reachability rules of § 4.6 (`no_link_semantics`).

Experimental Environment. All static analyses and concrete builds were executed on a workstation equipped with an Intel Core i7-7700 CPU (8 logical cores) and 31 GiB of RAM running Debian GNU/Linux, with Python 3.14.7, Z3 5.1.0, GNU Make 4.4.1, and GCC 16.2.0. Linux configurations that enable Rust support were compiled with `rustc` 1.95.0 and `bindgen` 0.72.1, and coreboot was compiled using its bundled i386 cross-compilation toolchain. Each static analysis run executes as a single-threaded process. For whole-tree performance measurements, we run each tool three times per subject on an idle system (load average below 1.0) and report the median wall-clock running time and peak resident set size (RSS).

6.2 RQ1: Build Agreement

6.3 RQ2: Scalability

6.4 RQ3: Developer Tasks

6.5 RQ4: Comparison and Ablation

7 Related Work

8 Conclusion

Data-Availability Statement

AI Use

(counts toward the 20-page limit)

References

[1] Thanassis Avgerinos, Alexandre Rebert, Sang Kil Cha, and David Brumley. 2014. Enhancing Symbolic Execution with Veritesting. In *ACM/IEEE International Conference on Software Engineering (ICSE)*. 1083–1094. doi:10.1145/2568225.2568293

[2] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 337–340. doi:10.1007/978-3-540-78800-3_24

[3] Paul Gazzillo. 2017. Kmax: Finding All Configurations of Kbuild Makefiles Statically. In *ESEC/FSE*. doi:10.1145/3106237.3106283

[4] Volodymyr Kuznetsov, Johannes Kinder, Stefan Bucur, and George Candea. 2012. Efficient State Merging in Symbolic Execution. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 193–204.

[5] ThanhVu Nguyen and KimHao Nguyen. 2020. Using Symbolic Execution to Analyze Linux KBuild Makefiles. In *IEEE International Conference on Software Maintenance and Evolution (ICSME), New Ideas and Emerging Results*.

[6] Koushik Sen, George Necula, Liang Gong, and Wontae Choi. 2015. MultiSE: Multi-path Symbolic Execution using Value Summaries. In *ESEC/FSE*. 842–853. doi:10.1145/2786805.2786830

[7] Ahmed Tamrawi, Hoan Anh Nguyen, Hung Viet Nguyen, and Tien N. Nguyen. 2012. Build Code Analysis with Symbolic Evaluation. In *ACM/IEEE International Conference on Software Engineering (ICSE)*. 650–660. doi:10.1109/ICSE.2012.6227152

[8] Eric Walkingshaw, Christian Kästner, Martin Erwig, Sven Apel, and Eric Bodden. 2014. Variational Data Structures: Exploring Tradeoffs in Computing with Variability. In *ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward!)*. 213–226. doi:10.1145/2661136.2661143